

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释（WHY + HOW），并以递进式优化路线（naive → tiling → vectorize → tensor core → warp specialized）展示 GPU 性能调优思维。

37 个 Kernel · 10 个 Phase · 全中文注释

项目主页：<https://github.com/xlite-dev/LeetCUDA>

更新日期：2026 年 6 月 16 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA), 涵盖:
6 // - 面试高频 CUDA kernel 的完整实现 (共 37 个 kernel)
7 // - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 // - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 // - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 // Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 // Phase 1 — 基础原语: Warp Reduce / Block Reduce (含 broadcast 增强版)
14 // Phase 2 — Elementwise: ReLU / Elementwise Add (基础 + float4 向量化)
15 // Phase 3 — Softmax: naive → safe(2-pass) → online(1-pass) + RMS/Layer Norm
16 // Phase 4 — GEMV: SGEMV K32/K128/K16 + HGEMV K32/K128/K16 (warp-per-row)
17 // Phase 5 — GEMM : SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
18 // Phase 6 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
19 // Phase 7 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
20 // Phase 8 — 杂项: Dot Product / Block All Reduce / Histogram
21 // Phase 9 — FlashAttention split_q (FA-2, 含 online softmax + P0V 寄存器复用)
22 //
23 // =====
24 //
25 // =====
26 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
27 // =====
28 //
29 // ---- GPU 架构速查 ----
30 //
31 // SM (Streaming Multiprocessor) 内部结构:
32 // - Warp Scheduler ×4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
33 // - Register File: 每 SM 65536 × 32-bit = 256KB
34 // - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
35 // - Tensor Cores: Hopper 每 SM 4 个, Blackwell 每 SM 8 个
36 // - Warp = 32 threads: 最小调度单元, SIMT 执行模型
37 //
38 // Memory Hierarchy 带宽数量级 (H100 参考):
39 // HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
40 // L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
41 // L1/SMEM: ~19 TB/s (每 SM ~228KB)
42 // Register: ~0 延迟, ~100+ TB/s 等效带宽
43 //
44 // 关键瓶颈判断:
45 // Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
46 // Compute-bound: AI 足够大, 受限于计算吞吐
47 // Latency-bound: 线程不够多, 无法隐藏内存延迟
48 //
49 // Occupancy 公式:
50 // occupancy = active_warps / max_warps_per_SM
51 // 受限于: register/thread × threads/block + shared_memory/block
52 //
53 // ---- 常见优化手段速查清单 ----
54 //
55 // 1. Coalesced Memory Access (合并访问)
56 // - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
57 // - 否则产生多次事务 (最坏 32 次)
58 //
59 // 2. Tiling (分块)
60 // - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
61 // - GEMM: Block Tile (BM×BN) + K Tile (BK)
62 //
63 // 3. Vectorized Memory Access (向量化)
64 // - 使用 float4/half2 等向量类型, 减少 load/store 指令数
65 // - float4 = 128-bit, 单条指令加载 16 bytes
66 //
67 // 4. Thread Tile (寄存器分块)
68 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
69 // - 减少线程总数, 降低同步开销
70 //
71 // 5. Bank Conflict Avoidance
72 // - Shared memory 有 32 banks × 4 bytes
73 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
74 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
75 //
76 // 6. Pipeline / Double Buffering (流水线)
77 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
78 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
79 //
80 // 7. Tensor Core (MMA / WGMMMA)
81 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
82 // - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
83 //
84 // 8. Warp Specialization (Hopper+)
85 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
86 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
87 //
88 // 9. TMA (Tensor Memory Accelerator, Hopper+)
89 // - 硬件 DMA 引擎, 支持 2D/3D 寻址, 零寄存器开销
90 // - 配合 cp.async.bulk 实现异步数据搬运
91 //
92 // ---- Roofline 分析公式 ----
93 //
94 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
95 //
96 // GEMM (M=N=K=4096):
97 // FLOPs = 2 × M × N × K = 2 × 4096³ 137 GFLOPS
98 // Bytes = (M×K + K×N + M×N) × sizeof(float) 200 MB
99 // AI 137G / 200M 685 FLOPS/Byte → compute-bound (远超 H100 的 ~50:1
100 // 墙)
101 //
102 // GEMV (M=4096, K=4096):
103 // FLOPs = 2 × M × K = 2 × 4096² 33 MFLOPS
104 // Bytes = (M×K + K + M) × sizeof(float) 67 MB
105 // AI 33M / 67M 0.5 FLOPS/Byte → severely memory-bound
106 //
107 // Softmax (N=4096): AI (5×N) / (2×N×4) 2.5 FLOPS/Byte → memory-bound
108 //
109 // =====
110 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
111 // =====
112 // 面试要点:
113 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
114 // memory
115 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
116 // 注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
117 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
118 //
119 #include <algorithm>
120 #include <cuda_fp16.h>
121 #include <cuda_runtime.h>
122 #include <float.h>
123 #include <stdio.h>
124 #include <stdlib.h>
125
126 #define WARP_SIZE 32
127 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
128 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
129 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
130
131 // =====
132 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
133 // =====
134 //
135 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
136 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
137 // 复杂度 O(logN), N=32 时仅需 5 步
138 // 双模板参数: T=数据类型, kWarpSize=segment width (默认 32)
139 // 第四个参数 kWarpSize 是 segment width: 限制 shuffle 在同一 segment 内
140 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
141 template <typename T = float, const int kWarpSize = WARP_SIZE>

```

```

142 __device__ __forceinline__ T warp_reduce_sum(T val) {
143 #pragma unroll
144 for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
145     val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
146 }
147 return val;
148 }
149
150 // Warp Reduce Max — generic
151 template <typename T = float, const int kWarpSize = WARP_SIZE>
152 __device__ __forceinline__ T warp_reduce_max(T val) {
153 #pragma unroll
154 for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
155     val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
156 }
157 return val;
158 }
159
160 // Warp Reduce Sum — FP16 (保留特化版本, 用于 HGEMV)
161 template <const int kWarpSize = WARP_SIZE>
162 __device__ __forceinline__ half warp_reduce_sum_f16(half val) {
163 #pragma unroll
164 for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
165     val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
166 }
167 return val;
168 }
169
170 // =====
171 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp0)
172 // =====
173
174 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
175 // 两级归约流程:
176 // 1. 每个 warp 内做 warp_reduce_sum → 得到每个 warp 的一个值
177 // 2. warp leader (lane=0) 写入 shared memory
178 // 3. syncthreads 后, lane 0-NUM_WARPS-1 读取 shared memory
179 // 4. warp0 内再做一次 warp_reduce → 得到最终结果
180 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则只有 warp0 知道结果)
181 template <const int NUM_THREADS = 256>
182 __device__ float block_reduce_sum_f32(float val) {
183     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
184     int warp = threadIdx.x / WARP_SIZE;
185     int lane = threadIdx.x % WARP_SIZE;
186     static __shared__ float shared[NUM_WARPS];
187
188     float value = warp_reduce_sum<WARP_SIZE>(val);
189     if (lane == 0)
190         shared[warp] = value;
191     __syncthreads();
192     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
193     value = warp_reduce_sum<NUM_WARPS>(value);
194     // 关键: broadcast 结果到所有线程, 后续用 result
195     // 做除法等操作时所有线程都能拿到
196     value = __shfl_sync(0xffffffff, value, 0, 32);
197     return value;
198 }
199
200 // Block Reduce Max — FP32 (增强版, 带 broadcast)
201 template <const int NUM_THREADS = 256>
202 __device__ float block_reduce_max_f32(float val) {
203     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
204     int warp = threadIdx.x / WARP_SIZE;
205     int lane = threadIdx.x % WARP_SIZE;
206     static __shared__ float shared[NUM_WARPS];
207
208     float value = warp_reduce_max<WARP_SIZE>(val);
209     if (lane == 0)
210         shared[warp] = value;
211     __syncthreads();
212     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
213     value = warp_reduce_max<NUM_WARPS>(value);
214     value = __shfl_sync(0xffffffff, value, 0, 32);
215     return value;
216 }
217
218 // Block Reduce Sum — FP32 (notes-v1 原版, 保留用于兼容旧 kernel)
219 // 注意: 此版本没有 broadcast, 仅 warp0 持有最终结果
220 template <const int NUM_THREADS = 128>
221 __device__ __forceinline__ float block_reduce_sum(float val) {
222     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
223     int warp = threadIdx.x / WARP_SIZE;
224     int lane = threadIdx.x % WARP_SIZE;
225     static __shared__ float shared[NUM_WARPS];
226
227     val = warp_reduce_sum<WARP_SIZE>(val);
228     if (lane == 0)
229         shared[warp] = val;
230     __syncthreads();
231     val = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
232     val = warp_reduce_sum<NUM_WARPS>(val);
233     return val;
234 }
235
236 template <const int NUM_THREADS = 128>
237 __device__ __forceinline__ float block_reduce_max(float val) {
238     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
239     int warp = threadIdx.x / WARP_SIZE;
240     int lane = threadIdx.x % WARP_SIZE;
241     static __shared__ float shared[NUM_WARPS];
242
243     val = warp_reduce_max<WARP_SIZE>(val);
244     if (lane == 0)
245         shared[warp] = val;
246     __syncthreads();
247     val = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
248     val = warp_reduce_max<NUM_WARPS>(val);
249     return val;
250 }
251
252 // =====
253 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
254 // =====
255 // 面试要点:
256 // - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
257 // - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
258 // - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
259
260 // ---- ReLU: y = max(0, x) ----
261 // Grid: ((N + 255) / 256, 1, 1)
262 // Block: (256, 1, 1)
263 // source: LeetCUDA/kernels/relu/relu.cu
264 __global__ void relu(float *x, float *y, int N) {
265     int idx = blockIdx.x * blockDim.x + threadIdx.x;
266     if (idx < N)
267         y[idx] = fmaxf(0.0f, x[idx]);
268 }
269
270 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
271 // Block: (256, 1, 1)
272 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
273 // Grid: ((N + 255) / 256, 1, 1)
274 // Block: (64, 1, 1)
275 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
276 // source: LeetCUDA/kernels/relu/relu.cu
277 __global__ void relu_vec4(float *x, float *y, int N) {
278     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
279     if (idx < N) {
280         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
281         float4 reg_y;
282         reg_y.x = fmaxf(0.0f, reg_x.x);
283         reg_y.y = fmaxf(0.0f, reg_x.y);
284         reg_y.z = fmaxf(0.0f, reg_x.z);
285         reg_y.w = fmaxf(0.0f, reg_x.w);
286         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
287     }
288 }
289

```

```

290 // ---- Elementwise Add: c = a + b ----
291 // Grid: ((N + 255) / 256, 1, 1)
292 // Block: (256, 1, 1)
293 // source: LeetCode/kernels/elementwise/elementwise.cu
294 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
295     int idx = blockIdx.x * blockDim.x + threadIdx.x;
296     if (idx < N)
297         c[idx] = a[idx] + b[idx];
298 }
299
300 // Elementwise Add + float4 向量化
301 // Grid: ((N + 255) / 256, 1, 1)
302 // Block: (64, 1, 1), block(64)*4(float4)=256 元素/block
303 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
304 // source: LeetCode/kernels/elementwise/elementwise.cu
305 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
306     {
307         int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
308         if ((idx + 3) < N) {
309             float4 reg_a = FLOAT4(a[idx]);
310             float4 reg_b = FLOAT4(b[idx]);
311             float4 reg_c;
312             reg_c.x = reg_a.x + reg_b.x;
313             reg_c.y = reg_a.y + reg_b.y;
314             reg_c.z = reg_a.z + reg_b.z;
315             reg_c.w = reg_a.w + reg_b.w;
316             FLOAT4(c[idx]) = reg_c;
317         } else if (idx < N) {
318             for (int i = 0; (idx + i) < N; i++) {
319                 c[idx + i] = a[idx + i] + b[idx + i];
320             }
321         }
322     }
323 // =====
324 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
325 // =====
326 // 面试要点:
327 // - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
328 // online (1-pass, 增量更新)
329 // - Online Softmax 是 FlashAttention 的数学基础
330 // - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次
331 // (mean
332 // + variance)
333 // - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
334 // ---- Online Softmax 辅助结构 ----
335 // MD struct: 存储 running max (m) 和 running denominator (d)
336 // 算法来源: "Online normalizer calculation for softmax" (arXiv
337 // :1805.02867)
338 // 核心递推公式:
339 // m_new = max(m_old, m_cur)
340 // d_new = d_old * exp(m_old - m_new) + d_cur * exp(m_cur - m_new)
341 // FlashAttention 中用同样的公式做 online rescaling:
342 // 0_new = diag(exp(m_old - m_new)) * 0_old + exp(m_cur - m_new) * P@V
343 struct __align__(8) MD {
344     float m; // running max
345     float d; // running denominator (sum of exp(x - max))
346 };
347 // Warp Reduce for Online Softmax
348 // 与普通 reduce 不同: 归约时需同时更新 m 和 d (因为 max 在不断变大)
349 template <const int kWarpSize = WARP_SIZE>
350 __device__ __forceinline__ MD warp_reduce_md_op(MD value) {
351     unsigned int mask = 0xffffffff;
352 #pragma unroll
353     for (int stride = kWarpSize >> 1; stride >= 1; stride >>= 1) {
354         MD other;
355         other.m = __shfl_xor_sync(mask, value.m, stride);
356         other.d = __shfl_xor_sync(mask, value.d, stride);
357
358         bool value_bigger = (value.m > other.m);
359         MD bigger_m = value_bigger ? value : other;
360         MD smaller_m = value_bigger ? other : value;
361
362         // 关键: 更新 d 时需要 rescale 旧的 d 到新的 max 尺度下
363         value.d = bigger_m.d + smaller_m.d * __expf(smaller_m.m - bigger_m.m
364 );
365         value.m = bigger_m.m;
366     }
367 }
368 // =====
369 // Phase 3a: Softmax — 三级递进 (面试核心考点)
370 // =====
371 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
372 // 回答线索: naive(溢出) → safe(2-pass, 稳定) → online(1-pass, 为 FA 打基
373 // 础)
374
375 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定)----
376 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
377 // 问题: x 值很大时 exp(x) 溢出为 inf
378 template <const int NUM_THREADS = 256>
379 // Grid: (S, 1, 1), S=batch*seq_len,
380 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个
381 // block
382 // 处理一个 token
383 // source: LeetCode/kernels/softmax/softmax.cu
384 __global__ void softmax_f32_per_token_kernel(float *x, float *y, int N) {
385     const int tid = threadIdx.x;
386     const int idx = blockIdx.x * blockDim.x + tid;
387
388     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
389     float exp_sum = block_reduce_sum_f32<NUM_THREADS>(exp_val);
390     if (idx < N)
391         y[idx] = exp_val / exp_sum;
392 }
393 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定)----
394 // 面试重点: 为什么 Softmax 需要 Safe?
395 // - exp(89) 4.5e38 已接近 float32 上限, exp(100) 直接 inf
396 // - 减去 max 后: exp(x - max) exp(0) = 1.0, 永不超过 1
397 // - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
398 // - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
399 template <const int NUM_THREADS = 256>
400 // Grid: (S, 1, 1)
401 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
402 // source: LeetCode/kernels/softmax/softmax.cu
403 __global__ void safe_softmax_f32_per_token_kernel(float *x, float *y, int
404 N) {
405     const int tid = threadIdx.x;
406     const int idx = blockIdx.x * blockDim.x + tid;
407
408     // Pass 1: block reduce max — 找最大值
409     float val = (idx < N) ? x[idx] : -FLT_MAX;
410     float max_val = block_reduce_max_f32<NUM_THREADS>(val);
411
412     // Pass 2: exp(x - max) → block reduce sum
413     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
414     float exp_sum = block_reduce_sum_f32<NUM_THREADS>(exp_val);
415
416     if (idx < N)
417         y[idx] = exp_val / exp_sum;
418 }
419 // ---- Level 3: Online Safe Softmax (1-pass, FlashAttention 的数学基础)
420 // ----
421 // 面试重点: Online Softmax 为什么重要?
422 // - Safe Softmax 需要 3 遍遍历: 找 max → exp+sum → 除法
423 // - Online Softmax 用递推公式合并为 1 遍遍历
424 // - 核心思想: 处理新元素时, 用新旧 max 的差值 rescale 旧 denominator
425 // - 正是 FlashAttention 中 "online rescaling":
426 // 0_new = diag(exp(m_old - m_new)) * 0_old + exp(m_cur - m_new) * P@V
427 // 算法参考: "Online normalizer calculation for softmax" (arXiv
428 // :1805.02867)
429 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会
430 // 写回 y
431 template <const int NUM_THREADS = 256>

```

```

429 // Grid: (S, 1, 1)
430 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
431 // source: LeetCUDA/kernels/softmax/softmax.cu
432 __global__ void online_safe_softmax_f32_per_token_kernel(const float *x,
433                                                         float *y, int N)
434 {
435     int local_tid = threadIdx.x;
436     int global_tid = blockIdx.x * NUM_THREADS + threadIdx.x;
437     const int WARP_NUM = NUM_THREADS / WARP_SIZE;
438     int warp_id = local_tid / WARP_SIZE;
439     int lane_id = local_tid % WARP_SIZE;
440     // 初始化: 每个线程持有一个 (max, denom) 对
441     MD val;
442     val.m = global_tid < N ? x[global_tid] : -FLT_MAX;
443     val.d = global_tid < N ? 1.0f : 0.0f;
444
445     // Block reduce: warp_reduce_md_op 在归约中自动更新 m 和 d
446     __shared__ MD shared[WARP_NUM];
447     MD res = warp_reduce_md_op<WARP_SIZE>(val);
448
449     if (lane_id == 0)
450         shared[warp_id] = res;
451     __syncthreads();
452
453     // 第二级归约: warp0 收集各 warp 结果再做一次 md_op (复用 block_reduce 模式)
454     // 只有 local_tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个
455     // block 可见
456     if (local_tid < WARP_SIZE) {
457         MD block_res =
458             local_tid < WARP_NUM ? shared[local_tid] : MD{-FLT_MAX, 0.0f};
459         block_res = warp_reduce_md_op<WARP_NUM>(block_res);
460         if (local_tid == 0) {
461             shared[0] = block_res;
462         }
463     }
464     __syncthreads();
465
466     // 用全局 max 和 denom 做最终 softmax
467     MD final_res = shared[0];
468     float d_total_inverse = __fdivdef(1.0f, final_res.d);
469     // 边界线即使看到 d=0 的填充值, 也不会走到写回路径
470     if (global_tid < N) {
471         y[global_tid] = __expf(x[global_tid] - final_res.m) * d_total_inverse;
472     }
473 }
474
475 // =====
476 // Phase 3b: RMS Normalization (1-pass reduce)
477 // =====
478 // 面试要点:
479 // - RMS Norm:  $y = x / \text{rms}(x) * g$ ,  $\text{rms}(x) = \sqrt{\text{mean}(x^2)}$ 
480 // - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
481 // - Llama 系列使用 RMS Norm
482 // - grid(N, K/K), block(K): 一行一个 block
483
484 template <const int NUM_THREADS = 128>
485 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
486 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
487 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
488 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
489     int tid = threadIdx.x;
490     int bid = blockIdx.x;
491     int idx = bid * blockDim.x + threadIdx.x;
492     const float epsilon = 1e-5f;
493
494     __shared__ float s_variance;
495     float value = (idx < N * K) ? x[idx] : 0.0f;
496     float variance = value * value;
497     variance = block_reduce_sum<NUM_THREADS>(variance);
498     if (tid == 0)
499         s_variance = rsqrtf(variance / (float)K + epsilon);
500     __syncthreads();
501     if (idx < N * K)
502         y[idx] = (value * s_variance) * g;
503 }
504
505 // RMS Norm + float4
506 template <const int NUM_THREADS = 128 / 4>
507 // Grid: (N, 1, 1)
508 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
509 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
510 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
511 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K)
512 {
513     int tid = threadIdx.x;
514     int bid = blockIdx.x;
515     int idx = (bid * blockDim.x + threadIdx.x) * 4;
516     const float epsilon = 1e-5f;
517     __shared__ float s_variance;
518     float4 reg_x = FLOAT4(x[idx]);
519     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y
520                                     +
521                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w
522                                     : 0.0f;
523     variance = block_reduce_sum<NUM_THREADS>(variance);
524     if (tid == 0)
525         s_variance = rsqrtf(variance / (float)K + epsilon);
526     __syncthreads();
527     float4 reg_y;
528     reg_y.x = reg_x.x * s_variance * g;
529     reg_y.y = reg_x.y * s_variance * g;
530     reg_y.z = reg_x.z * s_variance * g;
531     reg_y.w = reg_x.w * s_variance * g;
532     if (idx < N * K)
533         FLOAT4(y[idx]) = reg_y;
534 }
535 // =====
536 // Phase 3c: Layer Normalization (2-pass reduce)
537 // =====
538 // 面试要点:
539 // - Layer Norm:  $y = (x - \text{mean}) / \text{std} * g + b$ 
540 // - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)^2)/K)
541 // - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
542
543 template <const int NUM_THREADS = 128>
544 // Grid: (N, 1, 1), 一行一个 block
545 // Block: (128, 1, 1), NUM_THREADS=K=128
546 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
547 __global__ void layer_norm(float *x, float *y, float g, float b, int N,
548                             int K) {
549     int tid = threadIdx.x;
550     int bid = blockIdx.x;
551     int idx = bid * blockDim.x + threadIdx.x;
552     const float epsilon = 1e-5f;
553
554     __shared__ float s_mean;
555     __shared__ float s_variance;
556     float value = (idx < N * K) ? x[idx] : 0.0f;
557
558     // Pass 1: compute mean
559     float sum = block_reduce_sum<NUM_THREADS>(value);
560     if (tid == 0)
561         s_mean = sum / (float)K;
562     __syncthreads(); // 必须等待 s_mean 对所有线程可见
563
564     // Pass 2: compute variance = (x - mean)^2
565     float variance = (value - s_mean) * (value - s_mean);
566     variance = block_reduce_sum<NUM_THREADS>(variance);
567     if (tid == 0)
568         s_variance = rsqrtf(variance / (float)K + epsilon);

```



```

568 __syncthreads(); // 必须等待 s_variance 对所有线程可见
569
570 if (idx < N * K)
571     y[idx] = ((value - s_mean) * s_variance) * g + b;
572 }
573
574 // Layer Norm + float4
575 template <const int NUM_THREADS = 128 / 4>
576 // Grid: (N, 1, 1)
577 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
578 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
579 // source: LeetCode/kernels/layer-norm/layer_norm.cu
580 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int
    N,
581                                int K) {
582     int tid = threadIdx.x;
583     int bid = blockIdx.x;
584     int idx = (bid * blockDim.x + threadIdx.x) * 4;
585     const float epsilon = 1e-5f;
586
587     __shared__ float s_mean;
588     __shared__ float s_variance;
589     float4 reg_x = FLOAT4(x[idx]);
590     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w)
        0.0f;
591
592     float sum = block_reduce_sum<NUM_THREADS>(value);
593     if (tid == 0)
594         s_mean = sum / (float)K;
595     __syncthreads();
596
597     float4 reg_x_hat;
598     reg_x_hat.x = reg_x.x - s_mean;
599     reg_x_hat.y = reg_x.y - s_mean;
600     reg_x_hat.z = reg_x.z - s_mean;
601     reg_x_hat.w = reg_x.w - s_mean;
602     float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y
        +
603                     reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
604     variance = block_reduce_sum<NUM_THREADS>(variance);
605     if (tid == 0)
606         s_variance = rsqrtf(variance / (float)K + epsilon);
607     __syncthreads();
608
609     float4 reg_y;
610     reg_y.x = reg_x_hat.x * s_variance * g + b;
611     reg_y.y = reg_x_hat.y * s_variance * g + b;
612     reg_y.z = reg_x_hat.z * s_variance * g + b;
613     reg_y.w = reg_x_hat.w * s_variance * g + b;
614     if (idx < N * K)
615         FLOAT4(y[idx]) = reg_y;
616 }
617
618 // =====
619 // Phase 4: GEMV — 矩阵向量乘 (纯 memory-bound 算子, warp-per-row 策略)
620 // =====
621 // 面试要点:
622 // - GEMV 是典型的 memory-bound 算子: AI O(1), 瓶颈在内存带宽
623 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
624 // - 不同 K 值对应不同分块策略:
625 //   K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
626 //   K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
627 //   K=16 < 32: 一个 warp 不够, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
628 //
629 // a: M×K, x: K×1, y: M×1, 计算: y = a * x
630
631 // ---- SGEMV K32: 基础 warp-per-row ----
632 // 设计: block(32, 4), blockDim.x=WARP_SIZE=K, blockDim.y=4 行/block
633 // grid(M/4), 每个 warp 负责一行
634 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
635 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
636 // Block: (32, 4, 1), 每行 1 warp
637 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
638 // source: LeetCode/kernels/sgemv/sgemv.cu
639
640 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
641     int tx = threadIdx.x; // 0-31
642     int ty = threadIdx.y; // 0-3
643     int bx = blockIdx.x;
644     int lane = tx % WARP_SIZE; // 0-31
645     int m = bx * blockDim.y + ty; // 全局行号
646     if (m < M) {
647         float sum = 0.0f;
648         int NUM_WARPS = (K + WARP_SIZE - 1) / WARP_SIZE;
649         #pragma unroll
650         for (int w = 0; w < NUM_WARPS; ++w) {
651             int k = w * WARP_SIZE + lane;
652             sum += a[m * K + k] * x[k];
653         }
654         sum = warp_reduce_sum<WARP_SIZE>(sum);
655         if (lane == 0)
656             y[m] = sum;
657     }
658
659 // ---- SGEMV K128: float4 向量化 ----
660 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
661 // Grid: ((M + 3) / 4, 1, 1)
662 // Block: (32, 4, 1)
663 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
664 // source: LeetCode/kernels/sgemv/sgemv.cu
665 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
666     int tx = threadIdx.x; // 0-31
667     int ty = threadIdx.y; // 0-3
668     int bx = blockIdx.x;
669     int lane = tx % WARP_SIZE; // 0-31
670     int m = blockDim.y * bx + ty;
671     if (m < M) {
672         float sum = 0.0f;
673         // K/128 个 warp 覆盖 K 维
674         int NUM_WARPS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
675         #pragma unroll
676         for (int w = 0; w < NUM_WARPS; ++w) {
677             int k = (w * WARP_SIZE + lane) * 4;
678             float4 reg_x = FLOAT4(x[k]);
679             float4 reg_a = FLOAT4(a[m * K + k]);
680             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
681                     reg_a.w * reg_x.w);
682         }
683         sum = warp_reduce_sum<WARP_SIZE>(sum);
684         if (lane == 0)
685             y[m] = sum;
686     }
687 }
688
689 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
690 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
691 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane
    处理
692 // row1
693 template <const int ROW_PER_WARP = 2>
694 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
695 // Block: (32, 4, 1)
696 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理
    2 行
697 // source: LeetCode/kernels/sgemv/sgemv.cu
698 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
699     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) /
        ROW_PER_WARP;
700     int tx = threadIdx.x;
701     int ty = threadIdx.y;
702     int bx = blockIdx.x;
703     int lane = tx % WARP_SIZE;
704     int k = lane % K_WARP_SIZE; // 0-15
705     int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
706     if (m < M) {
707         float sum = A[m * K + k] * x[k];
708         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
709         // 注意: 判断条件是 k == 0, 不是 lane == 0!
710     }

```

```

711     if (k == 0)
712         y[m] = sum;
713 }
714 }
715
716 // ---- HGEMV K32: FP16 版本 ----
717 // 策略与 SGEMV K32 完全一致, 仅数据类型从 float 变为 half
718 // Grid: ((M + 3) / 4, 1, 1)
719 // Block: (32, 4, 1)
720 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
721 // source: LeetCUDA/kernels/hgemv/hgemv.cu
722 __global__ void hgemv_k32_f16_kernel(half *a, half *x, half *y, int M,
    int K) {
723     int tx = threadIdx.x;
724     int ty = threadIdx.y;
725     int bx = blockIdx.x;
726     int lane = tx % WARP_SIZE;
727     int m = bx * blockDim.y + ty;
728     if (m < M) {
729         half sum = 0.0f;
730         int NUM_WARPS = (K + WARP_SIZE - 1) / WARP_SIZE;
731 #pragma unroll
732         for (int w = 0; w < NUM_WARPS; ++w) {
733             int k = w * WARP_SIZE + lane;
734             sum += a[m * K + k] * x[k];
735         }
736         sum = warp_reduce_sum_f16<WARP_SIZE>(sum); // FP16 warp reduce
737         if (lane == 0)
738             y[m] = sum;
739     }
740 }
741
742 // ---- HGEMV K128: FP16 + half2 向量化 ----
743 // 每个线程处理 4 个 half (2 个 half2), 一个 warp 覆盖 128 个元素
744 // Grid: ((M + 3) / 4, 1, 1)
745 // Block: (32, 4, 1)
746 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 half2 打包访问前提
747 // source: LeetCUDA/kernels/hgemv/hgemv.cu
748 __global__ void hgemv_k128_f16x4_kernel(half *a, half *x, half *y, int M,
    int K) {
749     int tx = threadIdx.x;
750     int ty = threadIdx.y;
751     int bx = blockIdx.x;
752     int lane = tx % WARP_SIZE;
753     int m = blockDim.y * bx + ty;
754
755     if (m < M) {
756         half sum = 0.0f;
757         int NUM_WARPS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
758 #pragma unroll
759         for (int w = 0; w < NUM_WARPS; ++w) {
760             int k = (w * WARP_SIZE + lane) * 4;
761             half2 reg_x_0 = HALF2(x[k + 0]);
762             half2 reg_x_1 = HALF2(x[k + 2]);
763             half2 reg_a_0 = HALF2(a[m * K + k + 0]);
764             half2 reg_a_1 = HALF2(a[m * K + k + 2]);
765             sum += (reg_x_0.x * reg_a_0.x + reg_x_0.y * reg_a_0.y +
766                 reg_x_1.x * reg_a_1.x + reg_x_1.y * reg_a_1.y);
767         }
768         sum = warp_reduce_sum_f16<WARP_SIZE>(sum);
769         if (lane == 0)
770             y[m] = sum;
771     }
772 }
773 }
774
775 // ---- HGEMV K16: FP16 + ROW_PER_WARP=2 ----
776 template <const int ROW_PER_WARP = 2>
777 // Grid: ((M + 7) / 8, 1, 1)
778 // Block: (32, 4, 1)
779 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理
    2 行
780 // source: LeetCUDA/kernels/hgemv/hgemv.cu
781 __global__ void hgemv_k16_f16_kernel(half *A, half *x, half *y, int M,
    int K) {
782     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) /

```

```

    ROW_PER_WARP;
783     int tx = threadIdx.x;
784     int ty = threadIdx.y;
785     int bx = blockIdx.x;
786     int lane = tx % WARP_SIZE;
787     int k = lane % K_WARP_SIZE;
788     int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
789     if (m < M) {
790         half sum = A[m * K + k] * x[k];
791         sum = warp_reduce_sum_f16<K_WARP_SIZE>(sum);
792         if (k == 0)
793             y[m] = sum;
794     }
795 }
796
797 // =====
798 // Phase 5: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
799 // =====
800 // 面试要点 (GEMM 优化五层金字塔):
801 // Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
802 // Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
803 // 个元素, 提高计算密度
804 // Level 3 — Vectorize (向量化访存): float4/half2, 减少
805 // load/store 指令数
806 // Level 4 — Tensor Core (MMA
807 // m16n8k16): 硬件矩阵乘单元, warp 级指令
808 // Level 5 — Warp Specialization +
809 // TMA (WGMMMA m64n128k16): Hopper 异步执行
810 //
811 // 计算密度递进:
812 // Level 1: AI B_K / (2*sizeof) 32/8 = 4 → 仍是 memory-bound
813 // Level 2: AI TM×TN×B_K / (2*sizeof) 8×8×8/8 = 64 → compute-bound
814 // Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
815
816 // =====
817 // Phase 5a: SGEMM + HGEMM (非 Tensor Core 路径)
818 // =====
819
820 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
821 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
822 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
823 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
824 // Block: (32, 32, 1)
825 // source: LeetCUDA/kernels/sgemm/sgemm.cu
826 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K)
    {
827     constexpr int BM = 32;
828     constexpr int BN = 32;
829     constexpr int BK = 32;
830     __shared__ float s_a[BM][BK], s_b[BK][BN];
831
832     int bx = blockIdx.x;
833     int by = blockIdx.y;
834     int tx = threadIdx.x;
835     int ty = threadIdx.y;
836     int tid = threadIdx.y * blockDim.x + tx;
837
838     // 线程到 smem 的映射: 32×32 线程, 每个线程加载 a 和 b 各 1 个元素
839     int load_smem_a_m = tid / 32;
840     int load_smem_a_k = tid % 32;
841     int load_smem_b_k = tid / 32;
842     int load_smem_b_n = tid % 32;
843     int load_gmem_a_m = by * BM + load_smem_a_m;
844     int load_gmem_b_n = bx * BN + load_smem_b_n;
845
846     float sum = 0.f;
847     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
848         int load_gmem_a_k = bk * BK + load_smem_a_k;
849         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
850         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
851         int load_gmem_b_k = bk * BK + load_smem_b_k;
852         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
853         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
854         __syncthreads(); // 确保整个 smem tile 加载完毕
855     }

```

```

856 #pragma unroll
857 for (int k = 0; k < BK; ++k) {
858     int comp_smem_a_m = load_smem_a_m;
859     int comp_smem_b_n = load_smem_b_n;
860     sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
861 }
862 __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
863 }
864 int store_gmem_c_m = load_gmem_a_m;
865 int store_gmem_c_n = load_gmem_b_n;
866 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
867 c[store_gmem_c_addr] = sum;
868 }
869
870 // ---- Level 2+3: SGEMM — Block Tile 128×128 + Thread Tile 8×8 + float4
871 // 三层 tile hierarchy: Block Tile → Thread Tile → K Tile
872 // BM=BN=128, TM=TN=8, BK=8, block(16, 16)=256 threads
873 // 每个线程计算 TM×TN=64 个元素, 大幅提升计算密度
874 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
875 // Block: (16, 16, 1), 256 线程
876 // source: LeetCUDA/kernels/sgemm/sgemm.cu
877 __global__ void sgemm_thread_tile_vec4(float *a, float *b, float *c, int
M,
int N, int K) {
878     constexpr int BM = 128;
879     constexpr int BN = 128;
880     constexpr int BK = 8;
881     constexpr int TM = 8;
882     constexpr int TN = 8;
883
884     int bx = blockIdx.x;
885     int by = blockIdx.y;
886     int tx = threadIdx.x;
887     int ty = threadIdx.y;
888     int tid = threadIdx.y * blockDim.x + tx;
889
890     // smem: 2×128×8×4 = 8KB (只有 1/4 的 L1/SMEM!)
891     __shared__ float s_a[BM][BK], s_b[BK][BN];
892
893     // 线程到 smem 的映射 — 256 线程协作加载 128×8 的矩阵
894     // s_a[BM][BK]=[128][8]: 每行 8 元素, 2 线程/行(float4)→ 256 线程覆盖
895     // 128 行
896     int load_smem_a_m = tid / 2;
897     int load_smem_a_k = (tid % 2 == 0) ? 0 : 4;
898     // s_b[BK][BN]=[8][128]: 每行 128 元素, 32 线程/行(float4)→ 256 线程覆盖
899     // 8 行
900     int load_smem_b_k = tid / 32;
901     int load_smem_b_n = (tid % 32) * 4;
902
903     int load_gmem_a_m = by * BM + load_smem_a_m;
904     int load_gmem_b_n = bx * BN + load_smem_b_n;
905
906     // 寄存器分块: 每个线程累积 TM×TN=64 个部分和
907     float r_c[TM][TN] = {0.0f};
908
909     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
910         int load_gmem_a_k = bk * BK + load_smem_a_k;
911         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
912         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
913
914         int load_gmem_b_k = bk * BK + load_smem_b_k;
915         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
916         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
917
918         __syncthreads();
919
920 #pragma unroll
921 for (int k = 0; k < BK; k++) {
922 #pragma unroll
923 for (int m = 0; m < TM; m++) {
924 #pragma unroll
925 for (int n = 0; n < TN; n++) {
926     int comp_smem_a_m = ty * TM + m;
927     int comp_smem_b_n = tx * TN + n;
928     r_c[m][n] += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
929 }
930 }
931 }
932
933 // 写回: float4 向量化 store
934 #pragma unroll
935 for (int m = 0; m < TM; ++m) {
936     int store_gmem_c_m = by * BM + ty * TM + m;
937 #pragma unroll
938 for (int n = 0; n < TN; n += 4) {
939     int store_gmem_c_n = bx * BN + tx * TN + n;
940     int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
941     FLOAT4(c[store_gmem_c_addr]) = FLOAT4(r_c[m][n]);
942 }
943 }
944 }
945
946 // ---- Level 2+3 (FP16): HGEMM — Thread Tile 8×8 + half2 向量化 ----
947 // FP16 版本: BM=BN=128, TM=TN=8, BK=8, block(16, 16)
948 // half2 向量化: 每次加载 2 个 half (32-bit) 到寄存器
949 // 面试注意: 这里的优化重点是 half2 访存向量化; 算术语义以当前 half 运算符实现为准
950 template <const int BM = 128, const int BN = 128, const int BK = 8,
const int TM = 8, const int TN = 8>
951 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
952 // Block: (16, 16, 1), 256 线程
953 // source: LeetCUDA/kernels/hgemm/naive/hgemm.cu
954 __global__ void hgemm_t_8x8_sliced_k_f16x4_kernel(half *a, half *b, half
*c,
int M, int N, int K) {
955     int bx = blockIdx.x;
956     int by = blockIdx.y;
957     int tx = threadIdx.x;
958     int ty = threadIdx.y;
959     int tid = threadIdx.y * blockDim.x + tx;
960
961     // FP16 smem: 2×128×8×2 = 4KB (是 FP32 的一半!)
962     __shared__ half s_a[BM][BK], s_b[BK][BN];
963
964     // 线程到 smem 映射与 FP32 版本相同
965     int load_smem_a_m = tid / 2;
966     int load_smem_a_k = (tid % 2 == 0) ? 0 : 4;
967     int load_smem_b_k = tid / 32;
968     int load_smem_b_n = (tid % 32) * 4;
969
970     int load_gmem_a_m = by * BM + load_smem_a_m;
971     int load_gmem_b_n = bx * BN + load_smem_b_n;
972
973     // 注意: 寄存器分块用 half 类型
974     half r_c[TM][TN] = {__float2half(0.0f)};
975
976     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
977         int load_gmem_a_k = bk * BK + load_smem_a_k;
978         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
979         // half2 向量化: 一次加载 2 个 half = 4 bytes
980         HALF2(s_a[load_smem_a_m][load_smem_a_k]) = HALF2(a[load_gmem_a_addr]);
981
982         int load_gmem_b_k = bk * BK + load_smem_b_k;
983         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
984         HALF2(s_b[load_smem_b_k][load_smem_b_n]) = HALF2(b[load_gmem_b_addr]);
985
986         __syncthreads();
987
988 #pragma unroll
989 for (int k = 0; k < BK; k++) {
990 #pragma unroll
991 for (int m = 0; m < TM; m++) {
992 #pragma unroll
993 for (int n = 0; n < TN; n++) {
994     int comp_smem_a_m = ty * TM + m;

```



```

996         int comp_smem_b_n = tx * TN + n;
997         r_c[m][n] += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
998     }
999 }
1000 }
1001 __syncthreads();
1002 }
1003
1004 #pragma unroll
1005 for (int m = 0; m < TM; ++m) {
1006     int store_gmem_c_m = by * BM + ty * TM + m;
1007 #pragma unroll
1008     for (int n = 0; n < TN; n += 2) {
1009         int store_gmem_c_n = bx * BN + tx * TN + n;
1010         int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1011         HALF2(c[store_gmem_c_addr]) = HALF2(r_c[m][n]);
1012     }
1013 }
1014 }
1015
1016 // =====
1017 // Phase 5b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
1018 // =====
1019 // 面试要点:
1020 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1021 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16×8] = [16×16]×[16×8]
1022 // - ldmatrix: 从 shared memory 加载 16×16 的矩阵片段到寄存器 (4 条 32-bit 寄存器)
1023 // 寄存器)
1024 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1025 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1026 //
1027 // TN 布局详解 (面试高频考点):
1028 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1029 // BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1030 // N = Normal (列优先, BLAS 原生格式)
1031 // T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1032 // 第一个字母 → A 的 op, 第二个字母 → B 的 op
1033 // 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对 BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1034 // 视角下: TN = A row-major [M×K], B col-major [N×K] 在 cuBLAS
1035 // 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1036 // T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1037 // N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1038 //
1039 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1040 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1041 // N = col-major (列优先, BLAS native = normal)
1042 //
1043 // NN 布局对比: C[M×N] = A[M×K] × B[K×N]
1044 // - A: row-major [M, K], B: row-major [K, N]
1045 // - 两者都是行优先 → BLAS 视角都是 T → cuBLAS: (T, T)
1046 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1047 //
1048 // TN 布局: C[M×N] = A[M×K] × B[N×K] (A 行优先, B 列优先)
1049 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1050 // - B [N×K]: col-major → 全局索引 B[n*K + k] (内维是 K!)
1051 // 物理含义: 存的是原 B[K×N] 的转置 B^T
1052 // - 优势: B 已是 col-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1053 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1054 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1055 //
1056 // WGMMA (Warp Group MMA, Hopper+):
1057 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1058 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (比 MMA 大 32 倍)
1059 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128 threads) 做计算
1060 // - TMA: 硬件 DMA, 零寄存器开销, 支持 2D/3D 寻址
1061 //
1062 // =====
1063 // Phase 5b-1: MMA PTX 宏定义
1064 // =====
1065 // ---- gmem → smem: cp.async ----
1066 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n"

```

```

1138 //
1139 // TN 布局在本 kernel 中的体现 (T=A 行优先, N=B 列优先):
1140 // - A[M][K]: row-major, 全局索引 A[m*K + k], shared memory s_a[BM][BK]
1141 // - B[N][K]: col-major, 全局索引 B[n*K + k] ( 注意内维是 K 不是 N!)
1142 // shared memory s_b[BN][BK] = s_b[N_tile][K_tile]
1143 // - ldmatrix A: 用 x4 (非转置), 因为 A 是 row-major, ldmatrix 原生匹配
1144 // - ldmatrix B: 用 x2 (非转置), 因为 B 已是 col-major (BLAS Normal), 无需
1145 // .trans
1146 // - MMA 指令: row.col = A row, B col → 天然匹配 TN 布局
1147 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block
           swizzle
1148 // - grid.z = S 个 swizzle 分区
1149 // - grid.x = 每个分区内部需要发射多少个 128x128 的 N tiles
1150 // - bx = blockIdx.z * gridDim.x + blockIdx.x, 把连续 block 打散到不同 N
           区域, 改善
1151 // L2 命中
1152 // Block: (256, 1, 1), 8 warps
1153 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1154 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K =
           16,
1155           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1156           const int WARP_TILE_M = 4, const int WARP_TILE_N = 4,
1157           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1158 __global__ void __launch_bounds__(256)
1159   hgemm_mma_m16n8k16_mma2x4_warp4x4_stages_dsmem_tn_kernel(half *A,
           half *B,
1160           half *C, int
           M,
1161           int N, int K) {
1162
1163   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1164   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx
           .x;
1165   const int by = blockIdx.y;
1166   constexpr int BM = MMA_M * MMA_TILE_M * WARP_TILE_M; // 16*2*4=128
1167   constexpr int BN = MMA_N * MMA_TILE_N * WARP_TILE_N; // 8*4*4=128
1168   constexpr int BK = MMA_K; // 16
1169
1170   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1171   // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B
           col-major)
1172   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会
           额外
1173   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict. 这里
           先保留最简
1174   // PAD=0 版本.
1175   extern __shared__ half smem[];
1176   half *s_a = smem;
1177   half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1178   constexpr int s_a_stage_offset = BM * BK; // 128*16
1179   constexpr int s_b_stage_offset =
           BN * BK; // 128*16 BN(128)*BK(16) = B col-major 的 smem 布局
1180
1181   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1182   const int warp_id = tid / WARP_SIZE; // 0-7
1183   const int lane_id = tid % WARP_SIZE; // 0-31
1184   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1185   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1186
1187   // 线程到 global memory 的映射 (用于加载 A 和 B)
1188   // TN 布局关键: A[m*K+k] 是 row-major, B[n*K+k] 是 col-major (内维是
           K!)
1189   int load_smem_a_m = tid / 2; // 0-127
1190   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1191   int load_smem_b_n = tid / 2; // 0-127 → B 的 N 方向 (col-major 的行)
1192   int load_smem_b_k =
           (tid % 2 == 0) ? 0 : 8; // 0, 8 → B 的 K 方向 (col-major 的列)
1193   int load_gmem_a_m = by * BM + load_smem_a_m;
1194   int load_gmem_b_n =
           bx * BN + load_smem_b_n; // B 全局列号 = N 方向的 tile 起始 + 线程偏
           移
1195   if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1196     return;
1197
1198   // 累加器: 每个 warp 计算 WARP_TILE_M×WARP_TILE_N=16 个 MMA tile
1199   uint32_t RC[WARP_TILE_M][WARP_TILE_N][2] = {}; // 初始化为 0
1200
1201   // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1202   uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1203   uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1204
1205   // 预加载前 (K_STAGE-1) 个 stage
1206   // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B 用 n*K+k(col-major)
1207   #pragma unroll
1208   for (int k = 0; k < (K_STAGE - 1); ++k) {
1209     int load_gmem_a_k = k * BK + load_smem_a_k;
1210     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1211     int load_gmem_b_k = k * BK + load_smem_b_k;
1212     int load_gmem_b_addr =
           load_gmem_b_n * K + load_gmem_b_k; // B: [n][k] col-major
1213     uint32_t load_smem_a_ptr =
           (smem_a_base_ptr +
1214            (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) *
           sizeof(half));
1215     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1216     uint32_t load_smem_b_ptr =
           (smem_b_base_ptr +
1217            (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) *
           sizeof(half));
1218     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1219     CP_ASYNC_COMMIT_GROUP();
1220   }
1221   const int NUM_K_TILES = div_ceil(K, BK);
1222   CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 等待前 (K_STAGE-2) 个 group 完成
1223   __syncthreads();
1224   // 主循环: K 维分块迭代
1225   #pragma unroll
1226   for (int k = (K_STAGE - 1); k < NUM_K_TILES; ++k) {
1227     // Stage 选择: 轮转方式 (round-robin)
1228     // 这里统一用 %K_STAGE, 而不是只在 s2/s4 时写成 &1 / &3:
1229     // 原始实现要兼容 s3 这类非 2 的幂 stage 数, 因此直接用 mod 最稳妥.
1230     int smem_sel = (k + 1) % K_STAGE; // 当前计算的 stage
1231     int smem_sel_next = k % K_STAGE; // 下一轮加载的 stage
1232     // 异步加载下一批数据到 smem_sel_next
1233     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B 的 gmem 地址用
           n*K+k (col-major, 内维是 K)
1234     int load_gmem_a_k = k * BK + load_smem_a_k;
1235     int load_gmem_a_addr =
           load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1236     int load_gmem_b_k = k * BK + load_smem_b_k;
1237     int load_gmem_b_addr =
           load_gmem_b_n * K + load_gmem_b_k; // B: col-major [n][k] 内维是
           K!
1238     uint32_t load_smem_a_ptr =
           (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1239            load_smem_a_m * BK + load_smem_a_k) *
           sizeof(half));
1240     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1241     uint32_t load_smem_b_ptr =
           (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1242            load_smem_b_n * BK + load_smem_b_k) *
           sizeof(half));
1243     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1244     CP_ASYNC_COMMIT_GROUP();
1245     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1246     // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major
1247     // B 用 x2 (非转置), 因为 B 已经是 col-major, 无需 .trans
1248     uint32_t RA[WARP_TILE_M][4];
1249     uint32_t RB[WARP_TILE_N][2];

```

```

1275
1276 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1277 #pragma unroll
1278 for (int i = 0; i < WARP_TILE_M; ++i) {
1279     int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1280     int lane_smem_a_m =
1281         warp_smem_a_m + lane_id % 16; // ldmatrix 用 16 个 lane
1282     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1283     uint32_t lane_smem_a_ptr =
1284         (smem_a_base_ptr +
1285          (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1286           lane_smem_a_k) *
1287            sizeof(half));
1288     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1289 }
1290 // ldmatrix.x2: 加载 B 的 k16n8 片段 (col-major B, 非转置)
1291 // 为什么不用 .trans? 因为 TN 布局下 B 已经是 col-major [N][K]
1292 // ldmatrix 默认期望 col-major 输入 → 直接匹配
1293 #pragma unroll
1294 for (int j = 0; j < WARP_TILE_N; ++j) {
1295     int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1296     int lane_smem_b_n =
1297         warp_smem_b_n + lane_id % 8; // ldmatrix B 用 8 个 lane
1298     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1299     uint32_t lane_smem_b_ptr =
1300         (smem_b_base_ptr +
1301          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1302           lane_smem_b_k) *
1303            sizeof(half));
1304     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1305 }
1306 // MMA compute: 发射 WARP_TILE_M * WARP_TILE_N 条 MMA 指令
1307 #pragma unroll
1308 for (int i = 0; i < WARP_TILE_M; ++i) {
1309     #pragma unroll
1310     for (int j = 0; j < WARP_TILE_N; ++j) {
1311         HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1312                  RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1313     }
1314 }
1315 // 等待当前 stage 的异步加载完成, 然后 sync
1316 CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1317 __syncthreads();
1318 }
1319 // 尾端处理: 最后 (K_STAGE-1) 个 stage 的计算 (无新数据加载)
1320 if ((K_STAGE - 2) > 0) {
1321     CP_ASYNC_WAIT_GROUP(0);
1322     __syncthreads();
1323 }
1324 {
1325     #pragma unroll
1326     for (int k = 0; k < (K_STAGE - 1); k++) {
1327         uint32_t RA[WARP_TILE_M][4];
1328         uint32_t RB[WARP_TILE_N][2];
1329         int stage_sel = ((NUM_K_TILES - (K_STAGE - 1) + k) % K_STAGE);
1330     }
1331     #pragma unroll
1332     for (int i = 0; i < WARP_TILE_M; ++i) {
1333         int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1334         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1335         int lane_smem_a_k = (lane_id / 16) * 8;
1336         uint32_t lane_smem_a_ptr =
1337             (smem_a_base_ptr + (stage_sel * s_a_stage_offset +
1338                                lane_smem_a_m * BK + lane_smem_a_k) *
1339              sizeof(half));
1340         LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3],
1341                    lane_smem_a_ptr);
1342     }
1343 }
1344 }
1345 #pragma unroll
1346 for (int j = 0; j < WARP_TILE_N; ++j) {
1347     int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1348     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1349     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1350     uint32_t lane_smem_b_ptr =
1351         (smem_b_base_ptr + (stage_sel * s_b_stage_offset +
1352                             lane_smem_b_n * BK + lane_smem_b_k) *
1353          sizeof(half));
1354     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1355 }
1356 #pragma unroll
1357 for (int i = 0; i < WARP_TILE_M; ++i) {
1358     #pragma unroll
1359     for (int j = 0; j < WARP_TILE_N; ++j) {
1360         HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1361                  RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1362     }
1363 }
1364 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1365 {
1366     for (int i = 0; i < WARP_TILE_M; ++i) {
1367         uint32_t RCO[WARP_TILE_N][4];
1368         uint32_t RC1[WARP_TILE_N][4];
1369         // 用 warp shuffle 收集同一个 MMA tile 内不同 lane 的结果。
1370         // 对 m16n8k16 的 C fragment:
1371         // - lane 0/4/8/.../28 各自持有某一行里的 {c0,c1}
1372         // - lane+1 持有同一行里的 {c2,c3}
1373         // - lane+2 持有同一行里的 {c4,c5}
1374         // - lane+3 持有同一行里的 {c6,c7}
1375         // 所以每个 4-lane 子组可以通过 shfl 收齐一整行 8 个 half, 然后由
1376         // lane%4==0 的那个线程做一次 128-bit store。
1377         lane%4==0
1378         #pragma unroll
1379         for (int j = 0; j < WARP_TILE_N; ++j) {
1380             RCO[j][0] = RC[i][j][0];
1381             RC1[j][0] = RC[i][j][1];
1382             RCO[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1383             RCO[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1384             RCO[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1385             RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1386             RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1387             RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1388         }
1389         // 每 4 个 lane 中只有 lane 0 做 128-bit store
1390         if (lane_id % 4 == 0) {
1391             int store_warp_smem_c_m = warp_m * (MMA_M * WARP_TILE_M) + i *
1392             MMA_M;
1393             int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id
1394             / 4;
1395             #pragma unroll
1396             for (int j = 0; j < WARP_TILE_N; ++j) {
1397                 int store_warp_smem_c_n = warp_n * (MMA_N * WARP_TILE_N) + j *
1398                 MMA_N;
1399                 int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1400                 int store_gmem_c_addr_0 =
1401                     store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1402                 int store_gmem_c_addr_1 =
1403                     (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1404                 // 128-bit store: 一次写入 8 个 half
1405                 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1406                 *reinterpret_cast<float4*>(&RCO[j][0]);
1407                 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1408                 *reinterpret_cast<float4*>(&RC1[j][0]);
1409             }
1410         }
1411     }
1412 }

```

```
1413
1414 // =====
1415 // Phase 5b-3: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (
      Hopper)
1416 // =====
1417 // 面试要点 (WGMMMA vs MMA 对比):
1418 // - MMA: warp 级 (32 threads), 同步执行
1419 // - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-
      forget)
1420 // - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA
      的 32
1421 // 倍)
1422 // - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
1423 // - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过
      barrier
1424 // 同步
1425 // - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
1426
1427 // ---- WGMMMA 辅助函数 ----
1428 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "
      memory")
1429 #define WGMMMA_COMMIT_GROUP()
1430     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
1431 #define WGMMMA_WAIT_GROUP(n)
1432     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :::"n"(n) : "memory")
1433
1434 // Shared memory descriptor encode: 将 smem 地址编码为 WGMMMA 可用的描述符
1435 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4)
1436
1437 // make_smem_desc: 创建 WGMMMA 的 shared memory 矩阵描述符
1438 // 这里沿用原始 hgemm_wgmma 实现里的描述符约定:
1439 // - base addr: 当前 shared memory tile 基址
1440 // - leading offset: 16 bytes = 8 个 half * 2B, 对应 WGMMMA 在 tile 内沿
      minor
1441 // 方向前进一次的 byte 步长
1442 // - stride offset: 1024 bytes = 64 * 8 * 2B, 对应跨到下一条 major stripe
      的 byte 步长
1443 // - bit 62: 打开 128B swizzle
1444 // 这些字段是当前 WGMMMA/TMA 布局下的实现常量, 不要把它直接背成 BM/BK
1445 // 的原始字节数公式。
1446 __device__ inline uint64_t make_smem_desc(half *ptr) {
1447     uint32_t addr = static_cast<uint32_t>((__cvta_generic_to_shared(ptr)));
1448     uint64_t desc = 0x0000000000000000;
1449     desc |= SMEM_DESC_ENCODE(addr);
1450     desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16; // leading dim offset
      bytes
1451     desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32; // stride dim offset
      bytes
1452     desc |= 1llu << 62; // 128B swizzle
1453     return desc;
1454 }
1455
1456 // ---- WGMMMA PTX 指令宏 ----
1457 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
1458 // m64n128k16: M=64, N=128, K=16
1459 // f16.f16.f16: A=f16, B=f16, D=f16 (accumulation in f16)
1460 // 32 个输出寄存器 = 64×128/2/4 = 1024 个 half / 32 = 32 个 uint32
1461 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成)
1462 // ScaleD: 0=clear accum, 1=accumulate (用于 K 维迭代时累加)
1463 #define WGMMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB,
      TransA,
1464     TransB)
1465 {
1466     uint64_t desc_a = make_smem_desc(&(sA)[0]);
1467     uint64_t desc_b = make_smem_desc(&(sB)[0]);
1468     asm volatile(
1469         "{\n"
1470         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 "
1471         "{%0, %1, %2, %3, %4, %5, %6, %7, "
1472         "%8, %9, %10, %11, %12, %13, %14, %15, "
1473         "%16, %17, %18, %19, %20, %21, %22, %23, "
1474         "%24, %25, %26, %27, %28, %29, %30, %31}, "
1475         "%32, "
1476         "%33, "
1477         "%34, %35, %36, %37, %38;\n"
1478         "\n"
1479         : "r"((d)[0][0]), "r"((d)[0][1]), "r"((d)[0][2]), "r"((d)
      [0][3]),
1480         "r"((d)[1][0]), "r"((d)[1][1]), "r"((d)[1][2]), "r"((d)
      [1][3]),
1481         "r"((d)[2][0]), "r"((d)[2][1]), "r"((d)[2][2]), "r"((d)
      [2][3]),
1482         "r"((d)[3][0]), "r"((d)[3][1]), "r"((d)[3][2]), "r"((d)
      [3][3]),
1483         "r"((d)[4][0]), "r"((d)[4][1]), "r"((d)[4][2]), "r"((d)
      [4][3]),
1484         "r"((d)[5][0]), "r"((d)[5][1]), "r"((d)[5][2]), "r"((d)
      [5][3]),
1485         "r"((d)[6][0]), "r"((d)[6][1]), "r"((d)[6][2]), "r"((d)
      [6][3]),
1486         "r"((d)[7][0]), "r"((d)[7][1]), "r"((d)[7][2]), "r"((d)
      [7][3])
1487         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)),
1488         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA
      )),
1489         "n"(int32_t(TransB)));
1490 }
1491
1492 // ---- TMA Shared Memory Layout ----
1493 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[
      BK×BN]
1494 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
1495     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
1496     alignas(128)
1497         half B[BK * BN * QSIZE]; // B tile: row-major [BK, BN] (即 col-
      major B^T)
1498 };
1499
1500 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
1501 // 面试重点 — Warp Specialization:
1502 // WG0 (128 threads): Producer — 用 TMA 异步加载 A/B 到 shared memory
1503 // WG1 (128 threads): Consumer — 用 WGMMMA 做矩阵乘
1504 // 同步: cuda::barrier (CTA 级别), Producer 发 full 信号, Consumer 发
      empty
1505 // 信号 K_STAGE=3: 3 个 stage, Consumer 滞后 Producer 最多 2 步
1506 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block
      swizzle
1507 // Block: (256, 1, 1), 2 warpgroups(Producer+Consumer)
1508 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
1509 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
1510     const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
1511     const int BK = 64, const int NUM_THREADS = 256, const int
      K_STAGE = 3,
1512     const bool BLOCK_SWIZZLE = false>
1513 __global__ void __launch_bounds__(NUM_THREADS)
1514     hgemm_wgmma_m64n128k16_f16acc_stages_tma_ws_tn_kernel(
1515         int M, int N, int K, half *C,
1516         const CUtensorMap *__restrict__ tensorMapA,
1517         const CUtensorMap *__restrict__ tensorMapB) {
1518
1519     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
1520     // 对 row-major [H,W] 矩阵, TMA shape 参数写的是 (W,H) 而不是 (H,W),
1521     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主
      体,
1522     // 不展开宿主侧 create_tensor_map 细节。
1523
1524     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx
      .x;
1525     const int by = blockIdx.y;
1526     constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
1527     constexpr int B_WG_M = BM / num_consumers; // 128
1528
1529     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
1530         return;
1531
1532     extern __shared__ __align__(128) uint8_t smem[];
1533     WgmmaSMem<BM, BN, BK, K_STAGE> &s =
1534         *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE>*>(smem);
1535     half *s_a = s.A;
1536     half *s_b = s.B;
```



```

1537
1538 // CTA barrier: 同步 Producer 和 Consumer
1539 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
1540 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
1541
1542 const int num_blocks_k = K / BK;
1543 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
1544 const int tid = threadIdx.x % 128; // 0-127 within warpgroup
1545
1546 // 初始化 barriers (仅 thread 0 执行)
1547 if (threadIdx.x == 0) {
1548     for (int i = 0; i < K_STAGE; ++i) {
1549         // 这里的 128 不是“warp 数”也不是“线程块总线程数”，
1550         // 而是这个 barrier 上每轮会 arrive 的参与者总数：128 个 consumer 线程 + 1 个
1551         // producer 提交线程。
1552         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1
1553         producer
1554         init(&empty[i], num_consumers * 128 + 1); // same
1555     }
1556     cuda::device::experimental::fence_proxy_async_shared_cta();
1557 }
1558 __syncthreads();
1559
1560 // ===== Producer Warpgroup (WG0) =====
1561 if (wg_idx == 0) {
1562     if (tid == 0) { // 仅 producer 的 thread 0 执行 TMA 加载
1563         // TMA 是硬件 DMA 提交指令：发起一次 descriptor+坐标提交后，
1564         // 后续搬运由硬件异步完成，
1565         // 不需要整个 producer warpgroup 里的 128 个线程都参与拷贝。
1566         int qidx = 0;
1567         for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1568             ++block_k_iter, ++qidx) {
1569             if (qidx == K_STAGE)
1570                 qidx = 0;
1571
1572             // 等待 Consumer 释放此 stage (empty 信号)
1573             empty[qidx].wait(empty[qidx].arrive());
1574
1575             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
1576             cuda::device::experimental::
1577             cp_async_bulk_tensor_2d_global_to_shared(
1578                 &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
1579                 full[qidx]);
1580
1581             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
1582             cuda::device::experimental::
1583             cp_async_bulk_tensor_2d_global_to_shared(
1584                 &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
1585                 full[qidx]);
1586
1587             // 通知 Consumer 此 stage 已准备好 (full 信号)
1588             cuda::device::barrier_arrive_tx(full[qidx], 1,
1589                 (BK * BN + BK * BM) * sizeof(half));
1590         }
1591     }
1592 }
1593 // ===== Consumer Warpgroup (WG1) =====
1594 else {
1595     // Consumer 初始时“准备就绪”，arrive 到所有 empty barriers
1596     for (int i = 0; i < K_STAGE; ++i) {
1597         empty[i].arrive();
1598     }
1599
1600     // 累加器: B_WG_M/WGMMMA_M=2 行 × WGMMMA_N/16=8 列 = 16 组寄存器
1601     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
1602
1603     int qidx = 0;
1604     for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1605         ++block_k_iter, ++qidx) {
1606         if (qidx == K_STAGE)
1607             qidx = 0;
1608
1609         // 等待 Producer 的 full 信号
1610         full[qidx].wait(full[qidx].arrive());
1611
1612         // WGMMMA 指令序列的常见记法是：
1613         // FENCE -> 发射一串 WGMMMA -> COMMIT_GROUP -> WAIT_GROUP -> 下一轮再
1614         // FENCE
1615         // fence 是为后续 WGMMMA 建立可见性/顺序关系；commit 表示当前这组
1616         // WGMMMA
1617         // 已经发完；
1618         // wait 才是真正等待已 commit 的异步矩阵乘完成。
1619         // WGMMMA fence: 确保 smem 写可见、accum 寄存器准备好
1620         WGMMMA_FENCE();
1621
1622         // M 维迭代: BM/WGMMMA_M = 128/64 = 2
1623         #pragma unroll
1624         for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1625             half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
1626
1627             // K 维迭代: BK/WGMMMA_K = 64/16 = 4
1628             #pragma unroll
1629             for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
1630                 WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
1631                     s_b + qidx * BK * BN + k_it *
1632                     1, // ScaleD=1: accumulate (不清零)
1633                     1, 1, 0, 0);
1634             }
1635         }
1636
1637         WGMMMA_COMMIT_GROUP();
1638         WGMMMA_WAIT_GROUP(0); // 等待所有 WGMMMA 完成
1639
1640         // 释放此 stage (发 empty 信号给 Producer)
1641         empty[qidx].arrive();
1642     }
1643 }
1644
1645 // ===== Epilogue: 写回 C =====
1646 // WGMMMA m64n128k16 的输出寄存器映射可这样记：
1647 // row = warp * 16 + lane / 4
1648 // col = g * 16 + 2 * (lane % 4)
1649 // d[m_it][g][0] -> (row, col)
1650 // d[m_it][g][1] -> (row + 8, col)
1651 // d[m_it][g][2] -> (row, col + 8)
1652 // d[m_it][g][3] -> (row + 8, col + 8)
1653 // 所以每个线程一次写回 4 个 half2，刚好覆盖当前 16x16 子块里的四个象限
1654 // 位置。
1655 const int lane = tid % 32;
1656 const int warp = tid / 32;
1657 const int row = warp * 16 + lane / 4;
1658 half *block_C = C + by * BM * N + bx * BN;
1659
1660 #pragma unroll
1661 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1662     int yo = m_it * WGMMMA_M;
1663     #pragma unroll
1664     for (int g = 0; g < WGMMMA_N / 16; ++g) {
1665         int col = g * 16 + 2 * (lane % 4);
1666         // 将 uint32 直接 reinterpret 为 half2 pair 写入 C
1667         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
1668             d[m_it][g][0];
1669         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
1670             d[m_it][g][1];
1671         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
1672             d[m_it][g][2];
1673         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col +
1674             8]) =
1675             d[m_it][g][3];
1676     }
1677 }
1678 }
1679 }
1680 // =====
1681 // Phase 6: RoPE — 旋转位置编码 (Rotary Position Embedding)

```



```
1675 // =====
1676 // 面试要点:
1677 // - RoPE 数学公式: 对每对相邻维度做 2D 旋转
1678 // [x1']   [cos()  -sin()] [x1]
1679 // [x2'] = [sin()   cos()] [x2]
1680 // - _i = 1 / (theta^(2i/d)), theta=10000.0f (Llama 风格)
1681 // - token_pos = idx / N: token 在序列中的位置
1682 // - token_idx = idx % N: token 内的维度对索引
1683 // - 输入 [seq_len, hidden_size], 输出同形状
1684
1685 // Grid: ((seq_len * N + 255) / 256, 1, 1)
1686 // Block: (256, 1, 1)
1687 // source: LeetCUDA/kernels/rope/rope.cu
1688 __global__ void rope_f32_kernel(float *x, float *out, int seq_len, int N)
1689 {
1690     int idx = blockIdx.x * blockDim.x + threadIdx.x;
1691     float x1 = x[idx * 2];
1692     float x2 = x[idx * 2 + 1];
1693     int token_pos = idx / N; // 序列位置
1694     int token_idx = idx % N; // 维度对索引
1695
1696     // 频率计算: _i = 1 / (10000^(2i/d))
1697     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
1698     float sin_v = sinf(token_pos * exp_v);
1699     float cos_v = cosf(token_pos * exp_v);
1700
1701     // 2D 旋转
1702     float out1 = x1 * cos_v - x2 * sin_v;
1703     float out2 = x1 * sin_v + x2 * cos_v;
1704     out[idx * 2] = out1;
1705     out[idx * 2 + 1] = out2;
1706 }
1707 // =====
1708 // Phase 7: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
1709 // =====
1710 // 面试要点 (Bank Conflict 专题):
1711 // - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
1712 // - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
1713 // - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
1714 // - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
1715 //
1716 // 转置的三步演进:
1717 // naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
1718 // shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
1719 // BCF: smem 布局 [WARP_SIZE_S][WARP_SIZE_S*4+PAD], PAD=1 消除 bank
1720 // conflict
1721 // merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
1722
1723 // ---- Level 1: 基础版 (2D 索引, 合并读 + 非合并写)----
1724 // 每个线程处理 1 个元素, block(16,16)
1725 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取
1726 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 (32 次内存事务)
1727 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
1728 // Block: (16, 16, 1)
1729 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
1730 __global__ void mat_transpose_f32_row2col2d_kernel(float *x, float *y,
1731                                                     const int row,
1732                                                     const int col) {
1733     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
1734     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
1735     if (global_y < row && global_x < col) {
1736         // 读取: x[global_x][global_y] = x[global_x * col + global_y], 行优先, 合并
1737         // 写入: y[global_y][global_x] = y[global_y * row +
1738         // global_x], 列优先, 非合并
1739         y[global_y * row + global_x] = x[global_x * col + global_y];
1740     }
1741 }
1742 // ---- Level 4: 最佳版 (shared memory + bank conflict fix + merge write)
1743
1744 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
1745 // Block: (16, 16, 1)
```

```
1746 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
1747 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
1748 __global__ void
1749     mat_transpose_f32x4_shared_bcf_merge_write_row2col2d_kernel(
1750         float *x, float *y, const int row, const int col) {
1751     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
1752     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
1753     const int local_x = threadIdx.x;
1754     const int local_y = threadIdx.y;
1755
1756     constexpr int WARP_SIZE_S = 16;
1757     constexpr int PAD = 1;
1758     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
1759     __shared__ float tile[WARP_SIZE_S * 4][WARP_SIZE_S + PAD];
1760
1761     if (global_y * 4 < row && global_x < col) {
1762         // Step 1: 从 global memory 读取 4 行, 写入 shared
1763         // memory (行优先, 合并读取)
1764         float4 x_val;
1765         x_val.x = x[(global_y * 4) * col + global_x];
1766         x_val.y = x[(global_y * 4 + 1) * col + global_x];
1767         x_val.z = x[(global_y * 4 + 2) * col + global_x];
1768         x_val.w = x[(global_y * 4 + 3) * col + global_x];
1769         tile[local_y * 4][local_x] = x_val.x;
1770         tile[local_y * 4 + 1][local_x] = x_val.y;
1771         tile[local_y * 4 + 2][local_x] = x_val.z;
1772         tile[local_y * 4 + 3][local_x] = x_val.w;
1773         __syncthreads();
1774
1775         // Step 2: 从 shared memory 读取 ("转置"方向), 合并写入 global memory
1776         float4 smem_val;
1777         smem_val.x = tile[local_x * 4][local_y];
1778         smem_val.y = tile[local_x * 4 + 1][local_y];
1779         smem_val.z = tile[local_x * 4 + 2][local_y];
1780         smem_val.w = tile[local_x * 4 + 3][local_y];
1781
1782         const int gid_x = blockIdx.x * blockDim.x;
1783         const int gid_y = blockIdx.y * blockDim.y * 4;
1784         const int out_y = gid_y + local_x * 4;
1785         const int out_x = gid_x + local_y;
1786
1787         // float4 merge write: 1 次 128-bit store; 这里也隐含 out_y 为 4 的倍数
1788         *reinterpret_cast<float4*>(&y[(out_x * row + out_y) / 4]) =
1789             *reinterpret_cast<float4*>(&smem_val);
1790     }
1791 }
1792 // =====
1793 // Phase 8: 杂项 Ops — Dot Product / Block All Reduce / Histogram
1794 // =====
1795 // 面试要点:
1796 // - Dot Product: elementwise mul + reduce, 演示两种 reduce 模式结合
1797 // - Block All Reduce: 多 block 各自做 reduce, 最后 atomicAdd 到全局结果
1798 // - Histogram: atomicAdd 模式, 演示共享内存冲突的处理
1799
1800 // ---- Dot Product: y = sum(a[i] * b[i]) ----
1801 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
1802 template <const int NUM_THREADS = 128>
1803 // Grid: ((N + 127) / 128, 1, 1)
1804 // Block: (128, 1, 1)
1805 // source: LeetCUDA/kernels/dot-product/dot_product.cu
1806 __global__ void dot(float *a, float *b, float *y, int N) {
1807     int tid = threadIdx.x;
1808     int idx = blockIdx.x * NUM_THREADS + tid;
1809     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
1810     __shared__ float reduce_smem[NUM_WARPS];
1811
1812     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
1813     int warp = tid / WARP_SIZE;
1814     int lane = tid % WARP_SIZE;
1815
1816     prod = warp_reduce_sum<WARP_SIZE>(prod);
1817     if (lane == 0)
1818         reduce_smem[warp] = prod;
```

```

1819 __syncthreads();
1820
1821 prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
1822 if (warp == 0)
1823     prod = warp_reduce_sum<NUM_WARPS>(prod);
1824 if (tid == 0)
1825     atomicAdd(y, prod);
1826 }
1827
1828 // Dot Product + float4
1829 template <const int NUM_THREADS = 128 / 4>
1830 // Grid: ((N + 127) / 128, 1, 1)
1831 // Block: (32, 1, 1), 128/4=32
1832 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
1833 // source: LeetCUDA/kernels/dot-product/dot_product.cu
1834 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
1835     int tid = threadIdx.x;
1836     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
1837     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
1838     __shared__ float reduce_smem[NUM_WARPS];
1839
1840     float4 reg_a = FLOAT4(a[idx]);
1841     float4 reg_b = FLOAT4(b[idx]);
1842     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
1843         reg_a.z * reg_b.z + reg_a.w * reg_b.w)
1844         : 0.0f;
1845     int warp = tid / WARP_SIZE;
1846     int lane = tid % WARP_SIZE;
1847
1848     prod = warp_reduce_sum<WARP_SIZE>(prod);
1849     if (lane == 0)
1850         reduce_smem[warp] = prod;
1851     __syncthreads();
1852
1853     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
1854     if (warp == 0)
1855         prod = warp_reduce_sum<NUM_WARPS>(prod);
1856     if (tid == 0)
1857         atomicAdd(y, prod);
1858 }
1859
1860 // ---- Block All Reduce Sum: y = sum(a[0..N-1]) ----
1861 // 多 block 各自做 warp*smem*warp0 reduce, 然后 atomicAdd 到全局 y
1862 template <const int NUM_THREADS = 128>
1863 // Grid: ((N + 127) / 128, 1, 1)
1864 // Block: (128, 1, 1)
1865 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
1866 __global__ void block_all_reduce_sum(float *a, float *y, int N) {
1867     int tid = threadIdx.x;
1868     int idx = blockIdx.x * NUM_THREADS + tid;
1869     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
1870     __shared__ float reduce_smem[NUM_WARPS];
1871
1872     float sum = (idx < N) ? a[idx] : 0.0f;
1873     int warp = tid / WARP_SIZE;
1874     int lane = tid % WARP_SIZE;
1875
1876     sum = warp_reduce_sum<WARP_SIZE>(sum);
1877     if (lane == 0)
1878         reduce_smem[warp] = sum;
1879     __syncthreads();
1880
1881     sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
1882     if (warp == 0)
1883         sum = warp_reduce_sum<NUM_WARPS>(sum);
1884     if (tid == 0)
1885         atomicAdd(y, sum);
1886 }
1887
1888 // Block All Reduce Sum + float4
1889 template <const int NUM_THREADS = 128 / 4>
1890 // Grid: ((N + 127) / 128, 1, 1)
1891 // Block: (32, 1, 1), 128/4=32
1892 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
1893 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
1894 __global__ void block_all_reduce_sum_vec4(float *a, float *y, int N) {
1895     int tid = threadIdx.x;
1896     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
1897     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
1898     __shared__ float reduce_smem[NUM_WARPS];
1899
1900     float4 reg_a = FLOAT4(a[idx]);
1901     float sum = (idx < N) ? (reg_a.x + reg_a.y + reg_a.z + reg_a.w) : 0.0f;
1902     int warp = tid / WARP_SIZE;
1903     int lane = tid % WARP_SIZE;
1904
1905     sum = warp_reduce_sum<WARP_SIZE>(sum);
1906     if (lane == 0)
1907         reduce_smem[warp] = sum;
1908     __syncthreads();
1909
1910     sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
1911     if (warp == 0)
1912         sum = warp_reduce_sum<NUM_WARPS>(sum);
1913     if (tid == 0)
1914         atomicAdd(y, sum);
1915 }
1916
1917 // ---- Histogram: y[a[i]]++ ----
1918 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
1919 // Grid: ((N + 255) / 256, 1, 1)
1920 // Block: (256, 1, 1)
1921 // source: LeetCUDA/kernels/histogram/histogram.cu
1922 __global__ void histogram(int *a, int *y, int N) {
1923     int idx = blockIdx.x * blockDim.x + threadIdx.x;
1924     if (idx < N)
1925         atomicAdd(&y[a[idx]], 1);
1926 }
1927
1928 // Histogram + int4 向量化
1929 // Grid: ((N + 255) / 256, 1, 1)
1930 // Block: (64, 1, 1), 256/4=64
1931 // 注意: 该版本默认输入地址满足 int4 对齐; 最适合 N 按 4 对齐的场景
1932 // source: LeetCUDA/kernels/histogram/histogram.cu
1933 __global__ void histogram_vec4(int *a, int *y, int N) {
1934     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
1935     if (idx < N) {
1936         int4 reg_a = INT4(a[idx]);
1937         atomicAdd(&y[reg_a.x], 1);
1938         atomicAdd(&y[reg_a.y], 1);
1939         atomicAdd(&y[reg_a.z], 1);
1940         atomicAdd(&y[reg_a.w], 1);
1941     }
1942 }
1943
1944 // =====
1945 // Phase 9: FlashAttention-2 (Split-Q + MMA m16n8k16)
1946 // =====
1947 // 面试要点 (FlashAttention 算法):
1948 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
1949 // 但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写入
1950 // 2. FA 三板斧:
1951 // a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
1952 // b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
1953 // c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
1954 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
1955 // - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
1956 // - 优点: 减少 warp 间通信和 shuffle
1957 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691):
1958 // for each K,V tile:
1959 // S_cur = Q @ K^T * scale
1960 // m_new = max(m_old, row_max(S_cur))
1961 // P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
1962 // l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
1963 // O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
1964 // O_final = O_new / l_final
1965 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
1966 // - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
1967 // - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
1968 // 后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费

```

```
1969 // - 这是此实现的寄存器布局复用技巧，不要背成“所有 MMA A/C fragment
1970 // 都天然同构”的通用结论
1971 //
1972 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
1973 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
1974 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
1975 // Block: (128, 1, 1), kNumThreads=
1976 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
1977
1978 // ---- 寄存器填充辅助函数 ----
1979 template <typename T, int M, const int N, const int K = 2>
1980 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
1981 #pragma unroll
1982     for (int i = 0; i < M; ++i)
1983 #pragma unroll
1984         for (int j = 0; j < N; ++j)
1985 #pragma unroll
1986             for (int k = 0; k < K; ++k)
1987                 R[i][j][k] = val;
1988 }
1989
1990 template <typename T, int M, const int N = 2>
1991 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
1992 #pragma unroll
1993     for (int i = 0; i < M; ++i)
1994 #pragma unroll
1995         for (int j = 0; j < N; ++j)
1996             R[i][j] = val;
1997 }
1998
1999 // =====
2000 // FlashAttention-2 Split-Q Kernel (完整实现)
2001 // =====
2002 // Q,K,V,O: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
2003 //
2004 // Tile 设计 (以 kHeadDim=64 为例) :
2005 // Br = kMmaAtomM * kMmaTileSeqLenQ * kWarpTileSeqLenQ = 16*4*1 = 64
2006 // Bc = kMmaAtomN * kMmaTileSeqLenK * kWarpTileSeqLenK = 8*1*8 = 64
2007 // Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
2008 //
2009 // 执行流程:
2010 // 1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
2011 // 2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
2012 // 3) 3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
2013 // 4) 3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMM16816
2014 // 5) 3c: Online Safe Softmax — warp reduce max/row → exp(S*scale -
2015 //   max)
2016 // 6) 3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S
2017 //   做 A
2018 // 7) 3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
2019 // 8) 最终 rescale: O_final = (1/l_final) * O_new
2020 // 9) Epilogue: warp shuffle + 128-bit collective store
2021
2022 template <
2023     const int kHeadDim, // head dim: 32, 64, 128
2024     const int kMmaAtomM, // 16 (MMA instruction M dimension)
2025     const int kMmaAtomN, // 8 (MMA instruction N dimension)
2026     const int kMmaAtomK, // 16 (MMA instruction K dimension)
2027     const int kMmaTileSeqLenQ, // MMA tiles along Q's M dim, 4 → Br
2028     const int kMmaTileSeqLenK, // MMA tiles along K's N dim, 1 → Bc
2029     const int kMmaTileSeqLenP, // MMA tiles for P@V M dim, must equal
2030     const int kMmaTileHeadDimV, // MMA tiles for P@V N dim (head dim
2031     const int kWarpTileSeqLenQ, // warp tiles along Q's M, 1 → Br per
2032     const int kWarpTileSeqLenK, // warp tiles along K's N, 8 → Bc/warp
2033     const int kWarpTileSeqLenP, // warp tiles for P@V M dim, 1
2034
2035     const int kWarpTileHeadDimV, // warp tiles for P@V N dim,
2036     // kHeadDim/(8*kMmaTileHeadDimV)
2037     const int kStage, // pipeline stages for K: 1 or 2
2038     const int kPad> // padding for bank conflict avoidance
2039 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ *
2040     kMmaTileSeqLenK)
2041     flash_attn_mma_stages_split_q_kernel(half *Q, half *K, half *V, half
2042     *O,
2043     int QKV_seqlen, int QKV_head) {
2044
2045     // Tile dimensions
2046     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kWarpTileSeqLenQ; //
2047     64
2048     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kWarpTileSeqLenK; //
2049     64
2050     constexpr int kNumThreads =
2051     WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
2052     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
2053     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处
2054     // 理。
2055     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理
2056     // 了尾 tile。
2057     const float scale = 1.0f / sqrtf((float)kHeadDim);
2058
2059     // Block indexing
2060     const int QKV_batch_id = blockIdx.y / QKV_head;
2061     const int QKV_head_id = blockIdx.y % QKV_head;
2062     const int Q_tile_id = blockIdx.x;
2063     const int tid = threadIdx.x;
2064     const int warp_id = tid / WARP_SIZE;
2065     const int lane_id = tid % WARP_SIZE;
2066     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
2067     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
2068
2069     // Global memory base offsets for this (batch, head)
2070     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
2071     const int Q_gmem_offset =
2072     (QKV_batch_id * QKV_head + QKV_seqlen + QKV_head_id * QKV_seqlen) *
2073     kHeadDim;
2074     const int K_gmem_offset = Q_gmem_offset;
2075     const int V_gmem_offset = Q_gmem_offset;
2076     const int O_gmem_offset = Q_gmem_offset;
2077
2078     // Thread-to-smem mapping for cooperative load
2079     int load_smem_Q_Br = tid / (kNumThreads / Br);
2080     int load_smem_Q_d =
2081     (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
2082     int load_smem_K_Bc = tid / (kNumThreads / Bc);
2083     int load_smem_K_d =
2084     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2085     int load_smem_V_Bc = tid / (kNumThreads / Bc);
2086     int load_smem_V_d =
2087     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2088
2089     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
2090     if (load_gmem_Q_Br >= QKV_seqlen)
2091         return;
2092
2093     // ---- Shared memory layout ----
2094     extern __shared__ half smem[];
2095     constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+
2096     kPad]
2097     constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc,
2098     d+kPad]
2099     half *Q_tile_smem = smem;
2100     half *K_tile_smem = Q_tile_smem + Q_tile_size;
2101     half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage
2102     copies of K
2103     // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
2104     // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
2105     uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
2106     uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
2107     uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
```

```

2101 // ---- Online Softmax persistent state ----
2102 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
2103 // lane_block_row_sum_old[i][r]: running denominator 1 for row r of
    warp tile
2104 // i
2105 float lane_block_row_max_old[kWarpTileSeqLenQ][2];
2106 float lane_block_row_sum_old[kWarpTileSeqLenQ][2];
2107 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_block_row_max_old, -
    INFINITY);
2108 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
2109
2110 // ---- Register allocation ----
2111 uint32_t R_Q[kWarpTileSeqLenQ][4]; // Q regs
2112 uint32_t R_K[kWarpTileSeqLenK][2]; // K regs
2113 uint32_t R_V[kWarpTileHeadDimV][2]; // V regs
2114 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
2115 // 对单个 m16n8k16 tile 而言:
2116 // - reg[0] 持有该 tile 前 8 行里的两个 half 值
2117 // - reg[1] 持有该 tile 后 8 行里的两个 half 值
2118 // 后续 softmax、P0V、online rescale 都直接围绕这组 fragment 布局做寄存器
    内变换。
2119 uint32_t R_S[kWarpTileSeqLenQ][kWarpTileSeqLenK][2]; // S=Q*Q^T / P=
    softmax(S)
2120 uint32_t R_O[kWarpTileSeqLenP][kWarpTileHeadDimV][2]; // 0 for current
    tile
2121 uint32_t R_D[kWarpTileSeqLenP][kWarpTileHeadDimV]
    [2]; // 0 accumulator (final output)
2122
2123
2124 fill_3D_regs<uint32_t, kWarpTileSeqLenQ, kWarpTileSeqLenK, 2>(R_S, 0);
2125 fill_3D_regs<uint32_t, kWarpTileSeqLenP, kWarpTileHeadDimV, 2>(R_D, 0);
2126
2127 // =====
2128 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
2129 // =====
2130 {
2131     int load_gmem_Q_addr =
        Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
2132     uint32_t load_smem_Q_ptr =
        smem_Q_base_ptr +
        (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(
            half);
2133 #pragma unroll
2134 for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
2135     CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
2136     CP_ASYNC_COMMIT_GROUP();
2137 }
2138
2139 // =====
2140 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
2141 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从
    tile 0
2142 // 开始,
2143 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
2144 // =====
2145 if constexpr (kStage > 1) {
2146 #pragma unroll
2147 for (int stage = 0; stage < (kStage - 1); ++stage) {
2148     int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
2149     int load_gmem_K_addr =
        K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2150     uint32_t load_smem_K_ptr =
        smem_K_base_ptr +
        (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
            load_smem_K_d) *
            sizeof(half);
2151 #pragma unroll
2152 for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2153     CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i],
        16);
2154     CP_ASYNC_COMMIT_GROUP();
2155 }
2156 CP_ASYNC_WAIT_GROUP(kStage - 2);
2157 __syncthreads();
2158 }
2159
2160 // =====
2161 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
2162 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
2163 // =====
2164 #pragma unroll 1
2165 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
2166     int smem_sel = tile_K_seqlen % kStage;
2167     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
2168
2169     // ---- 3a: 异步加载 K/V tile (多 stage pipeline)----
2170     if constexpr (kStage > 1) {
2171         // 只有 kStage>1 才能真正做 K 的 pipeline:
2172         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮
            预取” 的 K
            // tile。
2173         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖
            同一块
            // smem。
2174         // Load current V tile (no pipeline for V — one stage is enough)
            {
2175             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
2176             int load_gmem_V_addr =
                V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
2177             uint32_t load_smem_V_ptr =
                smem_V_base_ptr +
                (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof
                    (half);
2178             #pragma unroll
2179             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2180                 CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i],
                    16);
2181             }
2182             CP_ASYNC_COMMIT_GROUP();
2183         }
2184
2185         // Prefetch next K tile (pipelined)
2186         if ((tile_K_seqlen + 1) < Tc) {
2187             int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
2188             int load_gmem_K_addr =
                K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2189             uint32_t load_smem_K_ptr =
                smem_K_base_ptr +
                (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim +
                    kPad) +
                    load_smem_K_d) *
                    sizeof(half);
2190             #pragma unroll
2191             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2192                 CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i],
                    16);
2193             }
2194             CP_ASYNC_COMMIT_GROUP();
2195         }
2196     }
2197
2198     // ---- 3b: Q*Q^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环
        ----
2199     fill_3D_regs<uint32_t, kWarpTileSeqLenQ, kWarpTileSeqLenK, 2>(R_S,
        0);
2200     #pragma unroll
2201     for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d)
        {
2202         // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
2203         #pragma unroll
2204         for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2205             int warp_smem_Q_Br =
                warp_QP * (kMmaAtomM * kWarpTileSeqLenQ) + i * kMmaAtomM;
2206             int lane_smem_Q_Br =
                warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
2207             int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; //
                0, 8
2208             uint32_t lane_smem_Q_ptr =
                smem_Q_base_ptr +

```



```

2233         (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof298
(half);
2234         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
2235             lane_smem_Q_ptr);
2236     }
2237
2238     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
2239     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
2240 #pragma unroll
2241     for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2242         int warp_smem_K_Bc =
2243             warp_KV * (kMmaAtomN * kWarpTileSeqLenK) + j * kMmaAtomN;
2244         int lane_smem_K_Bc =
2245             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
2246         int lane_smem_K_d =
2247             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
2248         uint32_t lane_smem_K_ptr =
2249             smem_K_base_ptr +
2250             (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad)
2251             +
2252             lane_smem_K_d) *
2253             sizeof(half);
2254         LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
2255     }
2256     // MMA: S[tile] += Q[tile] @ K^T[tile]
2257 #pragma unroll
2258     for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2259 #pragma unroll
2260         for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2261             MMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q
2262             [i][2],
2263             R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
2264             R_S[i][j][1]);
2265         }
2266     } // end loop over d
2267     __syncthreads();
2268
2269     // =====
2270     // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P
2271     // back to
2272     // R_S
2273     // =====
2274     // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
2275     // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0-7 里的两个 half 值 {c0,
2276     // c1}
2277     // - R_S[i][j][1] 对应 rows 8-15 里的两个 half 值 {c2,c3}
2278     // - lane 0-3 持有 row 0 的片段, lane 4-7 持有 row 1, ..., lane 28-31
2279     // 持有 row 7
2280     // - 对于 rows 8-15 也是同样的 lane 分组, 只是读取的是 reg[1]
2281     // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的
2282     // 是
2283     // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
2284     // Each (i, j) pair = one 16x8 MMA tile; there are kWarpTileSeqLenQ x
2285     // kWarpTileSeqLenK tiles.
2286
2287     float lane_row_max_new[kWarpTileSeqLenQ][2];
2288     float lane_row_sum_new[kWarpTileSeqLenQ][2];
2289     fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_row_max_new, -INFINITY
2290     );
2291     fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
2292
2293     // ---- Pass 1: Thread-level reduce max across all columns (
2294     // kWarpTileSeqLenK
2295     // tiles) ----
2296 #pragma unroll
2297     for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2298 #pragma unroll
2299         for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2300             // Extract half values from R_S registers (C matrix fragment
2301             layout)
2302             float2 t_reg_S_0 =
2303                 __half22float2(HALF2(R_S[i][j][0])); // rows 0-7: {c0, c1}
2304             float2 t_reg_S_1 =
2305                 __half22float2(HALF2(R_S[i][j][1])); // rows 8-15: {c2, c3}
2306             // S = (Q@K^T) * scale
2307             float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
2308             float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
2309             lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
2310             lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
2311         }
2312     // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
2313     // {0,4,8,...,28} hold valid data)
2314     lane_row_max_new[i][0] =
2315         warp_reduce_max<float, 4>(lane_row_max_new[i][0]);
2316     lane_row_max_new[i][1] =
2317         warp_reduce_max<float, 4>(lane_row_max_new[i][1]);
2318 }
2319
2320 // ---- Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
2321 // ----
2322 // 面试关键点: 这里将 P 写回 R_S 寄存器!
2323 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
2324 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重
2325 // 组。
2326 #pragma unroll
2327     for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2328         // m_new = max(m_old, m_cur)
2329         float block_row_max_new_0 =
2330             max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
2331         float block_row_max_new_1 =
2332             max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
2333
2334 #pragma unroll
2335         for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2336             float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
2337             float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
2338
2339             // P = exp(S * scale - m_new), 用 fma 保证精度
2340             t_reg_S_0.x =
2341                 __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
2342             t_reg_S_0.y =
2343                 __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
2344             t_reg_S_1.x =
2345                 __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
2346             t_reg_S_1.y =
2347                 __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
2348
2349             // Accumulate row sums
2350             lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
2351             lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
2352
2353             // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
2354             HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
2355             HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
2356         }
2357     // Warp-level reduce sum (warp_size = 4, same as max)
2358     lane_row_sum_new[i][0] =
2359         warp_reduce_sum<float, 4>(lane_row_sum_new[i][0]);
2360     lane_row_sum_new[i][1] =
2361         warp_reduce_sum<float, 4>(lane_row_sum_new[i][1]);
2362 }
2363
2364 __syncthreads();
2365
2366 // =====
2367 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = 0[Br,d]
2368 // =====
2369 // Wait for V to be ready before computing P@V
2370 if constexpr (kStage > 1) {
2371     if ((tile_K_seqlen + 1) < Tc) {
2372         CP_ASYNC_WAIT_GROUP(1);
2373     } else {
2374         CP_ASYNC_WAIT_GROUP(0);
2375     }
2376 } else {
2377     CP_ASYNC_WAIT_GROUP(0);
2378 }
2379 __syncthreads();

```



```

2371
2372     fill_3D_regs<uint32_t, kWarpTileSeqLenP, kWarpTileHeadDimV, 2>(R_0,
2373         0);
2374     // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
2375     // Bc=kMmaAtomK=16 + 1 iteration for kHeadDim 64 configurations
2376 #pragma unroll
2377     for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
2378         // ldmatrix.x2.trans: load V[Bc,d] with transposition
2379         // V is row-major [Bc,d], but NN matmul needs B matrix in col-major
2380         // transposed ldmatrix
2381 #pragma unroll
2382         for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2383             int warp_smem_V_d =
2384                 warp_KV * (kMmaAtomN * kWarpTileHeadDimV) + j * kMmaAtomN;
2385             int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
2386             int lane_smem_V_d = warp_smem_V_d;
2387             uint32_t lane_smem_V_ptr =
2388                 smem_V_base_ptr +
2389                 (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(
2390                     half);
2391             LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
2392         }
2393         // P matrix layout in R_S[i][j][2]:
2394         // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
2395         // | 64x64 | warp_KV 0 |
2396         // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
2397         // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
2398         // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
2399         // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
2400         // tile_V_Bc selects which 16-column slice of P to use:
2401         // tile_V_Bc=0 -> cols 0:16 -> MMA indices 0,1 -> w=0
2402         // tile_V_Bc=1 -> cols 16:32 -> MMA indices 2,3 -> w=2
2403         // tile_V_Bc=2 -> cols 32:48 -> MMA indices 4,5 -> w=4
2404         // tile_V_Bc=3 -> cols 48:64 -> MMA indices 6,7 -> w=6
2405         // 对应的 MMA A fragment 布局可以这样记:
2406         // - rows 0-7: lane 0-3 -> {a0,a1} 与 {a4,a5}, lane 4-7 -> 下一
2407         // 行, 依次类推
2408         // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以
2409         // 直接对接,
2410         // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对
2411         // 所有 MMA
2412         // fragment 都无条件成立的通用结论。
2413         int w = tile_V_Bc * 2;
2414 #pragma unroll
2415         for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2416             #pragma unroll
2417             for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2418                 HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
2419                     R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
2420                     R_V[j][0], R_V[j][1], // B fragment = V
2421                     R_0[i][j][0], R_0[i][j][1]); // C fragment output
2422             }
2423         } // end for tile_V_Bc
2424         __syncthreads();
2425         // =====
2426         // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
2427         // =====
2428         // 公式来源: FA2 paper Eq.(7-8)
2429         // 注意: FA2 论文中的公式有误 (取倒数而非 exp), 实际实现使用 exp(m_old
2430         // m_new)
2431 #pragma unroll
2432         for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2433             float block_row_max_new_0 = lane_row_max_new[i][0];
2434             float block_row_max_new_1 = lane_row_max_new[i][1];
2435             float block_row_sum_new_0 = lane_row_sum_new[i][0];
2436             float block_row_sum_new_1 = lane_row_sum_new[i][1];
2437
2438             float block_row_max_old_0 = lane_block_row_max_old[i][0];
2439             float block_row_max_old_1 = lane_block_row_max_old[i][1];
2440
2441             block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0
2442 );
2443             block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1
2444 );
2445
2446             // Handle first iteration: m_old = -inf, need to use m_new directly
2447             block_row_max_old_0 =
2448                 (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0
2449 );
2450             block_row_max_old_1 =
2451                 (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1
2452 );
2453
2454             float rescale_o_factor_0 =
2455                 __expf(block_row_max_old_0 - block_row_max_new_0);
2456             float rescale_o_factor_1 =
2457                 __expf(block_row_max_old_1 - block_row_max_new_1);
2458
2459             // Rescale 0_old + Add P@V in one fused step
2460 #pragma unroll
2461             for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2462                 // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
2463                 // reg[0] -> rows 0-7 的 {c0,c1}
2464                 // reg[1] -> rows 8-15 的 {c2,c3}
2465                 float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
2466                 float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
2467                 float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2468                 float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2469
2470                 // 0_new = exp(m_old - m_new) * 0_old + P@V (fused multiply-add)
2471                 t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x,
2472 t_reg_0_0.x);
2473                 t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y,
2474 t_reg_0_0.y);
2475                 t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x,
2476 t_reg_0_1.x);
2477                 t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y,
2478 t_reg_0_1.y);
2479
2480                 HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2481                 HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2482             }
2483
2484             // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
2485             float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
2486             float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
2487             lane_block_row_sum_old[i][0] = __fmaf_rn(
2488                 rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
2489             lane_block_row_sum_old[i][1] = __fmaf_rn(
2490                 rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
2491
2492             // Update m
2493             lane_block_row_max_old[i][0] = block_row_max_new_0;
2494             lane_block_row_max_old[i][1] = block_row_max_new_1;
2495         }
2496
2497         // Wait for next K tile to be ready in smem before next iteration
2498         if constexpr (kStage > 1) {
2499             if ((tile_K_seqlen + 1) < Tc) {
2500                 CP_ASYNC_WAIT_GROUP(0);
2501             }
2502             __syncthreads();
2503         } // end outer loop over K seqlen
2504         __syncthreads();
2505         // =====
2506         // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
2507         // =====
2508 #pragma unroll
2509         for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2510             float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);

```

```

2505     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
2506 #pragma unroll
2507 for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2508     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2509     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2510     t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
2511     t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
2512     t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
2513     t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
2514     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2515     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2516 }
2517 }
2518
2519 // =====
2520 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit
store
2521 // =====
2522 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0-3,
2523 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
2524 #pragma unroll
2525 for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2526 #pragma unroll
2527 for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2528     uint32_t R_Z[2][4];
2529     R_Z[0][0] = R_D[i][j][0];
2530     R_Z[1][0] = R_D[i][j][1];
2531     R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
2532     R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
2533     R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
2534     R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
2535     R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
2536     R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
2537
2538     // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
2539     if (lane_id % 4 == 0) {
2540         int store_warp_regs_0_Br =
warp_QP * (kMmaAtomM * kWarpTileSeqLenP) + i * kMmaAtomM;
2542         int store_lane_gmem_0_Br =
Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
2544         int store_warp_regs_0_d =
warp_KV * (kMmaAtomN * kWarpTileHeadDimV) + j * kMmaAtomN;
2546         int store_lane_gmem_0_d = store_warp_regs_0_d;
2547         int store_gmem_0_addr_0 = 0_gmem_offset +
(store_lane_gmem_0_Br + 0) * kHeadDim +
store_lane_gmem_0_d;
2549         int store_gmem_0_addr_1 = 0_gmem_offset +
(store_lane_gmem_0_Br + 8) * kHeadDim +
store_lane_gmem_0_d;
2553         // LDST128BITS = reinterpret_cast<float4*>
2554         *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
2555             *reinterpret_cast<float4*>(&R_Z[0][0]);
2556         *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
2557             *reinterpret_cast<float4*>(&R_Z[1][0]);
2558     }
2559 }
2560 }
2561 }
2562
2563 // =====
2564 // End of notes-v2.cu
2565 // =====
2566 // 本文件覆盖了面试中最高频的 CUDA kernel 考点 (37 个 kernel):
2567 // 基础原语: warp_reduce (O(logN) butterfly), block_reduce (两级:
2568 // warp→smem→warp0) 优化手段: coalescing, tiling, thread tile, vectorize
,
2569 // pipeline, tensor core Softmax 递进: naive → safe(2-pass) → online(1-
pass,
2570 // FA 基础) GEMM 五层金字塔: tiling → thread tile → vectorize → MMA(TN布
局)
2571 // → WGMM(warp spec) FlashAttention: split-Q + online softmax + R_S
2572 // 寄存器复用 P@V Bank Conflict: 原理 + PAD 解决方案 + 四步演进 Memory
2573 // Hierarchy: HBM → L2 → L1/SMEM → Register BLAS 布局约定:
2574 // N=col-major(Normal), T=row-major(Transposed), TN=A行B列
2575 // =====

```